

Integrating RevenueCat

Into a React Native (Expo) App — iOS & Android

RevenueCat · App Store Connect · Google Play Console · expo-iap

A practical, procedure-level integration guide with real dashboard click-paths and working code.

Covers both integration styles: RevenueCat-managed purchases (recommended for new apps) and Observer Mode (for apps with an existing IAP implementation).

Table of Contents

- 1. Overview: Two Ways to Integrate
- 2. Prerequisites
- 3. Part A — App Store Connect Setup
- 4. Part B — Google Play Console Setup
- 5. Part C — RevenueCat Dashboard Setup
- 6. Part D — Installing the SDK in Expo
- 7. Part E — Code: RevenueCat-Managed Purchases (Recommended)
- 8. Part F — Code: Observer Mode (Existing IAP Implementation)
- 9. Part G — Backend Webhooks
- 10. Part H — Testing on Real Devices
- 11. Part I — Common Pitfalls
- 12. Appendix — Full Setup Checklist

1. Overview: Two Ways to Integrate

RevenueCat can be integrated in one of two ways, and the choice affects both the RevenueCat dashboard configuration and the client code. Decide this first.

	RevenueCat-Managed (recommended)	Observer Mode
Who owns checkout?	RevenueCat SDK (<code>purchasePackage</code>)	Your own IAP code (e.g. <code>expo-iap</code>)
Best for	New apps, or apps not yet shipping IAP	Apps that already have a working native-store purchase flow and just want RevenueCat's cross-platform entitlement layer / analytics / webhooks on top, without touching the checkout path
Entitlement checks	<code>customerInfo.entitlements.active</code>	Same — RevenueCat still builds entitlement state from observed transactions
Server-side receipt	Handled by RevenueCat	Still done by your own backend (RevenueCat is not in that path)

verification		
Config flag	(default)	<pre>purchasesAreCompletedBy: { type: 'MY_APP', storeKitVersion: 'STOREKIT2' } at configure() time</pre>

Everything in Parts A–D (store & dashboard setup) applies to both approaches identically. Part E covers the managed flow; Part F covers Observer Mode.

2. Prerequisites

- An Expo project using **EAS Build** (a managed workflow, custom dev client, or bare workflow) — native modules like this cannot run inside Expo Go.
- An active Apple Developer Program membership (for iOS) and a Google Play Console developer account (for Android), both able to accept payments (Paid Apps Agreement / a Play Console billing profile).
- A RevenueCat account (free tier is sufficient to start): app.revenuecat.com.

3. Part A — App Store Connect Setup

A.1 Create the app & complete Agreements, Tax, and Banking

1. App Store Connect → **Apps** → + → create your app with its bundle ID.
2. Go to **Business** → **Agreements, Tax, and Banking**. Accept the **Paid Apps Agreement** (required — subscriptions cannot resolve at all in StoreKit, even in sandbox, until this is Active).
3. Add a **Bank Account** and complete the relevant **Tax Form** (e.g. a U.S. Form W-9 for a U.S. entity). Until the tax form is submitted (not just "in progress"), product lookups will fail with a generic, unhelpful error from the client SDK — there is no error message that will tell you this is the cause. Always check this page first if products silently fail to resolve.

Gotcha: if you see "*The Tax ID entered already exists*" when submitting a tax form, this is an Apple-side conflict (a duplicate EIN record somewhere in Apple's systems) — it cannot be fixed from the UI. Escalate via developer.apple.com/contact/finance/ → Subject: *Tax* → Category: *Tax information submissions*, stating the exact error and confirming no other Apple Developer account should hold this EIN.

A.2 Create the subscription group & subscriptions

1. Your app → **Monetization** → **Subscriptions** → create a **Subscription Group** (e.g. `APP_PLANS`) — all your tiers/durations live inside one group so users can only be subscribed to one at a time within it.
2. Inside the group, **Create a subscription** for each plan (e.g. `com.yourapp.pro.monthly`). Set reference name, duration, and localized display info.
3. For each subscription, set a price in **Subscription Pricing** — you can set one price and let Apple auto-generate equivalents in other territories/currencies, or customize per territory.
4. Submit metadata for review. Subscriptions can be tested in the **Sandbox** in any state except *Rejected* — you do not need to wait for "Approved" to test.

A.3 Create a Sandbox Tester

1. **Users and Access** → **Sandbox** → **Testers** → +. Use an email not tied to any real Apple ID.
2. On the test device: **Settings** → **App Store** → **Sandbox Account** (iOS 17+) — sign in there, *not* by signing out of your real Apple ID. Purchases made with this signed-in sandbox account inside your app will hit Apple's sandbox environment automatically.

A.4 (RevenueCat-managed only) Generate an In-App Purchase Key

RevenueCat needs this to validate iOS transactions server-side (independent of whether you're using StoreKit 1 or 2).

1. **Users and Access** → **Integrations** → **In-App Purchase** → generate a key.
2. Download the `.p8` key file immediately — it cannot be re-downloaded later. Note the **Key ID** and your **Issuer ID**.
3. Upload the key file + Key ID + Issuer ID into RevenueCat under your iOS app's settings (Part C).

4. Part B — Google Play Console Setup

B.1 Create the app & get a build into a testing track

1. Play Console → **Create app**.

2. You must upload at least one build to **any** track (Internal testing is fastest) before subscription products can be created — Play Console ties product creation to having a real app package on file.

B.2 Create subscriptions & base plans

1. **Monetize with Play** → **Products** → **Subscriptions** → **Create subscription**. Set the product ID (e.g. `pro_monthly`) — this cannot be changed later.
2. Inside the subscription, **Add a base plan** — set billing period (monthly/yearly) and whether it auto-renews.
3. Click **Set prices** or **Manage country/region availability** for the base plan.

Gotcha — the most common silent-failure cause on Android: a freshly-created base plan defaults to whatever region you happened to have selected when you clicked "Set prices" — often just your own country. If a base plan is only priced in, say, India, then *any* Play Store account whose billing country is not India will get an empty product list back from `fetchProducts()` — no error, just nothing. Always explicitly check **Manage country/region availability** and add every country you intend to sell in (or all 174+ available regions, using one reference-currency price that Play auto-converts per region).

Gotcha — regional price caps: some currencies have a hard price ceiling for auto-renewing subscriptions (e.g. Google enforces roughly ₩70–₩600,000 for KRW). If you bulk-set a price in USD and let Play auto-convert, a high-priced annual plan can overflow a smaller currency's cap and that single region will show a validation error blocking the whole save — find it via the region row showing a red error icon and adjust that one region's price down manually.

B.3 Service account & API access (needed for both server-side receipt validation and RevenueCat)

1. In **Google Cloud Console**, create (or reuse) a Service Account in the project linked to your Play Console developer account, and generate a JSON key for it.
2. In Play Console → **Users and permissions** → **Invite new users** (or manage an existing service account entry), grant it at minimum: *View financial data* and *Manage orders and subscriptions* permissions, scoped to your app.

3. Upload this same JSON key into RevenueCat under your Android app's settings (Part C) — RevenueCat uses it to validate purchases and receive real-time notifications.

B.4 Real-time developer notifications (recommended, optional)

1. **Monetize with Play** → **Monetization setup** → **Google Play Billing** → **Real-time developer notifications**.
2. Create a Cloud Pub/Sub topic, grant Google's publishing service account permission to publish to it, and enter the topic name here.
3. In RevenueCat's Android app settings, connect the same topic under **Google developer notifications**.

This step is optional — RevenueCat's own webhook (Part G) still receives events without it, just with slightly less real-time precision on Google's side.

5. Part C — RevenueCat Dashboard Setup

C.1 Create a project and add your apps

1. app.revenuecat.com → + **New Project**.
2. Inside the project → **Apps** → + → add an **App Store** app (paste your iOS bundle ID, upload the .p8 key + Key ID + Issuer ID from A.4) and a **Play Store** app (paste your Android package name, upload the service-account JSON from B.3).
3. Go to **API keys** — RevenueCat auto-generates one public SDK key per app. Copy the iOS key (app1_...) and the Android key (goog_...) — these go into your app config, never a secret key.

C.2 Create Entitlements

An **Entitlement** is the feature/tier your code checks for (e.g. "pro"), decoupled from any specific store product ID. Go to **Entitlements** → +, give it an identifier.

C.3 Create Products & attach them to Entitlements

1. **Products** → +. Add one product per store per plan, using the *exact same* product/subscription ID you created in App Store Connect / Play Console.

2. For each product, attach the relevant Entitlement (e.g. both `pro_monthly` (iOS) and `pro_monthly` (Android) attach to the "pro" entitlement).

C.4 (Managed mode only) Create an Offering

An **Offering** is the set of Packages you actually present in your paywall UI — lets you run A/B experiments or region-specific pricing without a client update. **Offerings** → + → add Packages, each pointing at one of your Products. Mark one Offering as **Current**.

Observer Mode does not need this — your own IAP code decides what to show, RevenueCat just watches. Skip C.4 for Observer Mode.

6. Part D — Installing the SDK in Expo

```
npx expo install react-native-purchases
# Observer Mode with your own purchase flow also needs the purchase library itself, e.g.:
npx expo install expo-iap
```

No config-plugin entry is required in `app.json` / `app.config.js` for basic purchases — `react-native-purchases` autolinks like any other native module. You must use a custom dev client or EAS Build; it will not run inside Expo Go.

Keep both SDK keys out of source control. Read them from `process.env` inside a dynamic `app.config.js` into `extra`, and set the corresponding EAS environment variables (`eas env:create`) so cloud builds pick them up too:

```
// app.config.js
export default {
  expo: {
    // ...
    extra: {
      revenueCatIosKey: process.env.REVENUECAT_IOS_API_KEY,
      revenueCatAndroidKey: process.env.REVENUECAT_ANDROID_API_KEY,
    },
  },
};
```

7. Part E — Code: RevenueCat-Managed Purchases (Recommended)

E.1 Configure the SDK once, early in app startup

```
import { Platform } from 'react-native';
import Constants from 'expo-constants';
import Purchases from 'react-native-purchases';

const REVENUECAT_API_KEY = Platform.select({
  ios: Constants.expoConfig?.extra?.revenueCatIosKey,
  android: Constants.expoConfig?.extra?.revenueCatAndroidKey,
});

export function initRevenueCat(userId) {
  Purchases.configure({
    apiKey: REVENUECAT_API_KEY,
    appUserID: userId, // pass your own user id so it matches your backend records
  });
}
```

E.2 Fetch Offerings and present a paywall

```
const offerings = await Purchases.getOfferings();
const current = offerings.current;

if (current) {
  // current.availablePackages: [{ identifier, packageType, product, ... }]
  return current.availablePackages;
}
```


E.3 Purchase a package

```
import { PurchasesError } from 'react-native-purchases';

async function buy(pkg) {
  try {
    const { customerInfo } = await Purchases.purchasePackage(pkg);
    if (customerInfo.entitlements.active['pro']) {
      // unlock the feature immediately
    }
  } catch (e) {
    if (!e.userCancelled) {
      Alert.alert('Purchase failed', e.message);
    }
  }
}
```

E.4 Check entitlement status anywhere in the app

```
const customerInfo = await Purchases.getCustomerInfo();
const isPro = typeof customerInfo.entitlements.active['pro'] !== 'undefined';
```

E.5 Restore purchases

```
async function restore() {
  const customerInfo = await Purchases.restorePurchases();
  const isPro = typeof customerInfo.entitlements.active['pro'] !== 'undefined';
}
```

8. Part F — Code: Observer Mode

Use this when you already have a working purchase flow (here shown with `expo-iap@~3.0.0`'s unified API) and only want RevenueCat watching in the background for cross-platform entitlement/analytics, without it touching checkout.

F.1 Configure RevenueCat in Observer Mode

```
import { Platform } from 'react-native';
import Constants from 'expo-constants';

const REVENUECAT_API_KEY = Platform.select({
  ios: Constants.expoConfig?.extra?.revenueCatIosKey,
  android: Constants.expoConfig?.extra?.revenueCatAndroidKey,
});

let _configured = false;

export function initRevenueCat(userId) {
  if (_configured || !userId || !REVENUECAT_API_KEY) return;
  // eslint-disable-next-line global-require
  const Purchases = require('react-native-purchases').default;
  const { PURCHASES_ARE_COMPLETED_BY_TYPE, STOREKIT_VERSION } = require('react-native-purchases');

  Purchases.configure({
    apiKey: REVENUECAT_API_KEY,
    appUserID: userId,
    purchasesAreCompletedBy: {
      type: PURCHASES_ARE_COMPLETED_BY_TYPE.MY_APP,
      storeKitVersion: STOREKIT_VERSION.STOREKIT_2, // ignored on Android; safe to share
    },
  });
  _configured = true;
}
```

Call this *alongside* your existing purchase-flow init (e.g. expo-iap's `initConnection()`), not instead of it. RevenueCat will independently attach to the same native billing client and observe transactions your own code completes.

F.2 Your own purchase flow keeps working unchanged

```
import * as iap from 'expo-iap';

// Fetch products directly from the store (expo-iap ~3.0.0's unified API)
const products = await iap.fetchProducts({ skus: ['pro_monthly'], type: 'subs' });

// Purchase – Android needs the base-plan offerToken from the fetched product
const product = products.find((p) => p.id === 'pro_monthly');
const offerToken = product?.subscriptionOfferDetailsAndroid?.[0]?.offerToken;

await iap.requestPurchase({
  request: {
    android: { skus: ['pro_monthly'], ...(offerToken ? { subscriptionOffers: [{ offerToken }] } ) },
    ios: { sku: 'pro_monthly' },
  },
  type: 'subs',
  willAutoRenew: true,
});
```

F.3 Verify the receipt on your own backend (unchanged by RevenueCat)

```
iap.purchaseUpdatedListener(async (purchase) => {
  if (Platform.OS === 'ios') {
    // getReceiptDataIOS() returns the classic base64 receipt your backend's
    // Apple verifyReceipt call expects – NOT the same as purchase.purchaseToken,
    // which carries the newer StoreKit 2 JWS representation instead.
    const receipt = await iap.getReceiptDataIOS();
    await api.post('/payments/iap/apple/verify', { receipt });
  } else {
    await api.post('/payments/iap/google/verify', {
      product_id: purchase.productId,
      purchase_token: purchase.purchaseToken,
    });
  }
  await iap.finishTransaction({ purchase, isConsumable: false });
});
```

Version-migration trap: expo-iap rewrote its API in ~v2.7/v3.0 into a single unified surface. If you're on an older tutorial or upgrading an existing app, watch for these renamed/removed members — calling the old names throws `TypeError: undefined is not a function`, which a broad `try/catch` will silently swallow and make every purchase look like "product not found" with no useful error at all:

Old (pre-unified API)	New (~v3.0.0)

<code>getSubscriptions({ skus })</code>	<code>fetchProducts({ skus, type: 'subs' })</code>
<code>product.subscriptionOfferDetails</code>	<code>product.subscriptionOfferDetailsAndroid</code>
<code>product.productId</code> (on a <i>Product</i> , not a <i>Purchase</i>)	<code>product.id</code>
<code>purchase.jwsRepresentation / .transactionReceipt</code>	<code>await iap.getReceiptDataIOS()</code> for the legacy base64 format; <code>purchase.purchaseToken</code> for the unified JWS
<code>requestPurchase({ type: 'subs', ... })</code>	same, plus a now-required top-level <code>willAutoRenew: boolean</code>

9. Part G — Backend Webhooks

G.1 Configure the webhook in RevenueCat

1. RevenueCat dashboard → **Integrations** → **Webhooks** → **Add new configuration**.
2. Set the URL to a backend endpoint (e.g. `https://api.yourapp.com/payments/webhook/revenuecat`). Add an **Authorization header value** — RevenueCat sends it back on every request so you can verify the call actually came from RevenueCat.
3. Leave **App** as "All apps" if one endpoint should handle both iOS and Android events.

G.2 Minimal handler (Python/FastAPI example)

```
@router.post("/webhook/revenuecat", include_in_schema=False)
async def revenuecat_webhook(request: Request):
    if request.headers.get("authorization") != REVENUECAT_WEBHOOK_AUTH:
        raise HTTPException(401, "bad auth")

    body = await request.json()
    event = body.get("event", {})
    event_type = event.get("type")
    app_user_id = event.get("app_user_id")  # your own user id, if passed as appUserID
    product_id = event.get("product_id")

    if event_type in ("INITIAL_PURCHASE", "RENEWAL", "PRODUCT_CHANGE", "UNCANCELLATION"):
        await grant_entitlement(app_user_id, product_id)
    elif event_type in ("CANCELLATION", "EXPIRATION"):
        await revoke_entitlement(app_user_id)

    return {"ok": True}
```

Passing your own database user ID as RevenueCat's `appUserID` (in `Purchases.configure()`) means `event.app_user_id` in the webhook payload is already your primary key — no lookup table needed to map RevenueCat's identity back to your user.

10. Part H — Testing on Real Devices

H.1 iOS

- Build via EAS (`eas build --platform ios --profile production` or a dev-client profile) and distribute via TestFlight, or run a dev-client build directly.
- On the device: sign into the **Sandbox Account** under Settings → App Store (not your main Apple ID). Purchases route to Apple's sandbox automatically.
- Subscriptions in "Waiting for Review"/"In Review" status are still purchasable in Sandbox.

H.2 Android

- Build an `.aab` (`eas build --platform android --profile production`) and upload it to at least the **Internal testing** track in Play Console.
- Add the tester's Google account to an email list attached to that track under **Testers**. Being in *some* tester list in your Play Console account is not enough — the specific list must be checked/attached to the specific track you're distributing from.

- Have the tester open the track's official **opt-in link** (`play.google.com/apps/internaltest/<track-id>`) and install/update from there — not via a generic Play Store search, which can silently serve the public Production build instead.
- To confirm exactly which build a device actually has installed (the visible "Version" string is often identical across many build numbers):

```
adb shell dumpsys package your.package.name | grep -E "versionCode|lastUpdateTime"
```

11. Part I — Common Pitfalls (Field-Tested)

Symptom	Likely cause
"Product not found" / empty product list on Android, in every region you've personally tested	Base plan country/region availability defaulted to one country — check Manage country/region availability on the base plan (§B.2)
Same symptom, persists after fixing pricing/regions and waiting	Client-side API mismatch from an <code>expo-iap</code> version upgrade — instrument with <code>console.log</code> around the fetch/purchase calls and read live via <code>adb logcat</code> rather than assuming it's a store-config issue (§F.3 table)
iOS products never resolve, on any build, any device	Paid Apps Agreement not Active, or a Tax Form still shows "Missing" under Agreements, Tax, and Banking — check this before anything else (§A.1)
Test device keeps installing the wrong build even via the correct opt-in link	Play Store track-serving/propagation delay — fully uninstall, restart the device, re-open the opt-in link, and verify the real <code>versionCode</code> via <code>adb</code> before retesting
<code>eas submit</code> fails non-interactively with a credentials prompt	Add <code>submit.production.ios.ascAppId</code> and/or <code>submit.production.android.track</code> to <code>eas.json</code> so it doesn't need to ask interactively

Appendix — Full Setup Checklist

#	Task	Where
1	Accept Paid Apps Agreement	App Store Connect
2	Add bank account + submit tax form	App Store Connect
3	Create subscription group + subscriptions	App Store Connect
4	Create a sandbox tester	App Store Connect
5	Generate In-App Purchase Key (.p8)	App Store Connect

6	Upload a build to a testing track	Google Play Console
7	Create subscriptions + base plans + global pricing	Google Play Console
8	Create/authorize a service account, grant Play Console permissions	Google Cloud + Play Console
9	(Optional) Configure real-time developer notifications	Google Play Console
10	Create RevenueCat project, add iOS + Android apps, copy SDK keys	RevenueCat
11	Upload .p8 key (iOS) and service-account JSON (Android)	RevenueCat
12	Create Entitlements	RevenueCat
13	Create Products, attach to Entitlements	RevenueCat
14	(Managed mode) Create an Offering with Packages	RevenueCat
15	Configure the webhook + auth header	RevenueCat
16	Install SDK, read keys from env, call <code>configure()</code>	App code
17	Implement purchase flow (managed or observer) + entitlement checks	App code
18	Implement the webhook handler	Backend
19	Add tester accounts to the Internal Testing track's tester list	Google Play Console
20	Test with a sandbox account (iOS) / internal tester (Android), verifying real installed build via <code>adb</code>	Both

This guide reflects a real integration pass against `expo-iap@~3.0.0` and `react-native-purchases`, including issues that only surface once you actually try to make a purchase on a real device — not just what compiles.